

Class Members (Operators, Finalizer, Nested, and Composite Types)

📝 Notes by Menna Reda

Operator overloading

يعني اي Operator overloading قبل ما نفهم يعني ايه تعالى نشوف الكام مثال دول:

```
int x = 1;
int y = 2;
int z = x + y; //Arithmetic operator "binary"
```

هنا انا اقدر اجمع الرقمين على بعض عادي باستخدام ال Arithmetic operator من غير اي مشاكل

```
int x = 1;
int y = 2;
bool m = x > y; //Comparison operator "binary"
```

هنا كمان قدرت استخدم Comparison operator بين المتغيرين عادي وهتمشي عادي مع اي datatype زي ال string, decimal وغيرهم من Predefined Types او Built-in Types

لو انا بق عايز اجمع two objects على بعض طبعا دا هيعمل error لان مينفعش اجمع اثنين objects على بعض مباشرة اقدر اجمع كل fields جواهم وهكذا طب لا انا عايز استعمل ال + operator لما اكون عايز اجمعهم على بعض من غير ما يحصل اي مشاكل هنا ال #C وفرتلنا ال operator overloading

ملحوظه الاتنين objects هنا من نفس النوع يعني من class واحد طبعا.

Operator overloading

User defined type that can overload predefined operators.

مثال:

```
internal class Program
{
    static void Main(string[] args)
    {

        var m1 = new Money(10);
        var m2 = new Money(20);
        Console.WriteLine($"m1: {m1.Amount}, m2: {m2.Amount}");
    }
}
```

```

        var m3 = m1 + m2;
        Console.WriteLine($"m3 = {m1.Amount} + {m2.Amount} = {m3.Amount}");
        Console.WriteLine(m1==m2);
        Console.WriteLine((++m1).Amount);

        Console.ReadKey();
    }
}

public class Money
{
    private decimal amount;
    public decimal Amount => this.amount;
    public Money(decimal value)
    {
        this.amount = Math.Round(value, 2);
    }

    public static Money operator + (Money m1, Money m2)
    {
        Money m3 = new Money(m1.Amount + m2.Amount);
        return m3;
    }

    public static Money operator -(Money m1, Money m2)
    {
        Money m3 = new Money(m1.Amount - m2.Amount);
        return m3;
    }

    public static Money operator ++(Money money)
    {
        var value = money.Amount;
        return new Money(++value);
    }
}

```

يبقى انا هنا استخدمت ال **operators** اللي بتخدمها مع ال **datatype** العاديه مع ال **objects**

Note:

في ال Comparison operator لازم اعمل الحاجه وعكسها يعني اي يعني لو هعمل ال > لازم اعمل ال <
 نفس الكلام مع $\geq$ و $\leq$ و $\neq$

Memory Management & Finalizer

Finalizer / destructor

في الـ .NET، الذاكرة تنقسم إلى نوعين من الموارد (Resources):

Managed Resources:

وهي الـ Objects العادية التي ننشئها داخل الـ #C. الـ Garbage Collector يعلم عنها كل شيء ويقوم بمسحها من الذاكرة تلقائياً بمجرد أن ينتهي استخدامها.

امتى الـ object بيتعمله destroy لما يخلص الـ scope اللي متعرف فيه يعني اما يتم تنفيذ كل الـ block اللي هو متعرف فيه ويطلع من block او الـ scope دا

مثال:

```
static void Main(string[] args)
{
    //Scope 1
    var person=new Person();

    {
        //Scope 2
        var p = new Person();
        p.Name = "Menna";
    }

    {
        //Scope 3
        var p = new Person();
        p.Name = "Ahmed";
    }
}
```

هنا كل object ليه scope خاص بيه واول ما يخلص الـ scope بتاعه الـ CG بيعينه او يعلمه انه قابل للازالة بعد كد هو بيمسح في وقت الـ CG هو اللي بيحدده.

→ Object flagged to be destroyed when end of its scope reached.

Garbage Collection → Automatic memory management

When object get disposed:

1. End of program execution (implicit).

2. Memory full (implicit).
3. CG.Collect (explicit). هنا يتم استدعائها من المبرمج.

Unmanaged Resources:

وهي موارد نفتحها من خارج بيئة الـ .NET والـ CLR (مثل: فتح ملف على الويندوز File Handles ، أو فتح اتصال بقاعدة بيانات، أو حجز سلك شبكة Network Sockets). الـ **Garbage Collector** لا يرى هذه الموارد ولا يعرف كيف يغلقها بنفسه!

لو فتحنا 1000 ملف ومقفلنهمش الـ memory هتتملي هيجصل Memory leak

الحل هو الـ Finalizer

الـ **Finalizer** او **Destructor** بنكتبه داخل الكلاس، الـ Garbage Collector يناديهها تلقائياً قبل أن يسمح الـ **Object** من الذاكرة نهائياً.

وظيفتها الأساسية: هي أن نقول للـ **Object**: "قبل أن تموت وتختفي، تأكد أنك قمت بإغلاق أي ملف خارجي أو اتصال قمت بفتحه" (Cleaning up unmanaged resources).

هنطبقه كد

```
internal class Program
{
    static void Main(string[] args)
    {
        var person=new Person();
        {
            //Scope 1
            var p = new Person();
            p.Name = "Menna";
        }
        {
            //Scope 2
            var p = new Person();
            p.Name = "Ahmed";
        }
    }
}

public class Person
{
    public string Name { get; set; }
    public Person()
```

```

{
    Console.WriteLine("This is person Constructor");
}
~Person()
{
    Console.WriteLine("This is person Destructor");
}
}

```

اذن ال destructor بيتكتب كد

```
~ClassName() { }
```

من غير اي Access modifiers او parameters ومبتعملوش overloading

و ال CG هو اللي بيحدد امتى هيتنقذ... ودا ممكن يسببلي مشاكل اول حاجه انا معرفش الحاجات اللي في ال memory هتتمسح امتى

تاني حاجه لما بعمل destructor دا بيبيطئ ال Garbage Collector

لو مفيش destructor ال Objects تموت وتمسح من الذاكرة من أول دورة لل GC فوراً.

لاكن لو في destructor ال GC لا يستطيع مسحها فوراً.. بل يضعها في طابور خاص اسمه **Finalization Queue**، ويتركها حية لـ دورة ثانية حتى يشتغل ال Finalizer الخاص بها. هذا يجعل ال Object يعيش في الذاكرة لفترة أطول ومعه كل ال Objects المرتبطة به!

مثال على ال Explicit cleaning:

```

internal class Program
{
    static void Main(string[] args)
    {
        MakeSomeGarbage();
        Console.WriteLine($"Memory used before
collection{GC.GetTotalMemory(false):N0}");
        GC.Collect(); //Explicit cleaning
        Console.WriteLine($"Memory used after
collection{GC.GetTotalMemory(true):N0}");
    }
    public static void MakeSomeGarbage()
    {
        Version v;
        for (int i = 0; i < 1000; i++)
        {
            v = new Version();
        }
    }
}

```

```
}  
  
}  
  
}
```

Nested and composite types

Nested types

يعني اي **Nested types**:

Type defined within a class.

مثلا اعرف class جوا class فيه ممكن اعمل حاجه زي كد لو انا بستخدم ال class دا داخل ال class الثاني بس ومش محتاج الكلاينت(كلاس ال program) يشوفه ع الفاضي.

بستفاد اي من اني احط كلاس جوا كلاس:

1. زي ما قلنا بخلي الكلاينت ميشوفش حاجات كتير ع الفاضي
2. ال class الداخلي يقدر ي access العناصر ال private للخارجي

مثلا لو عندي:

```
class A  
{  
    private int x;  
    class B  
    {  
        void method()  
        {  
            A a = new A();  
            a.x = 1; //access private member without any method or property  
        }  
    }  
}
```

The methods of B explicitly have access to private members in class A

مش لازم اعمل حاجة غير كد الا لما الاقي اني محتاج اعمل كد فعلا يعني مثلا ان class B مبستخدموش غير جوا A بس وكمان ال logic اللي فيه بسيط مش محتاج اني اعمله بره لوحده.

Composite types

Composition is a type of "Has-A" relationship between classes.

الـ Composition هو نوع من العلاقات بين الكلاسات، معناه إن كلاس بيمتلك كلاس تاني جواه كـ Property.

مثال عملي يجمع الـ Nested مع الـ Composition:

```
internal class Program
{
    static void Main(string[] args)
    {

        var employee = new Employee();
        Console.WriteLine($"PolicyId: {employee.insurance.PolicyId}
        CompanyName: {employee.insurance.CompanyName}");

    }

}

class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public Insurance insurance { get; set; }
    public Employee()
    {

        insurance = new Insurance { PolicyId=-1,CompanyName="N/A"};
    }
    public class Insurance
    {
        public int PolicyId { get; set; }
        public string CompanyName { get; set; }
    }

}

class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

دلوقتي انا عندي Class ال Employee و Class ال Insurance بينهم relationship بعرف ال relationship عن طرق ال

```
property public Insurance insurance { get; set; }
```

EF Core هنفهم دا كويس في

ف انا شايف ان مفيش اي class ثاني بيستخدم ال Insurance غير ال Employee ف بحطة جواه

بس في العموم اني اعمل حاجة زي كد مش دايمًا احسا حاجة الهدف هنا اننا نفهم يعني اي Nested and composite types.